Regularization & Normalization

&&

Харитон Пицик

2026-03-29

Клуб машинного обучения

В чём проблема?

Как мы уже ранее убедились, глубокие нейронные сети – это выдающиеся и дорогие модели, которые способны извлекать из данных сложные и нетривиальные паттерны. Однако за это есть своя плата – большой риск переобучения.

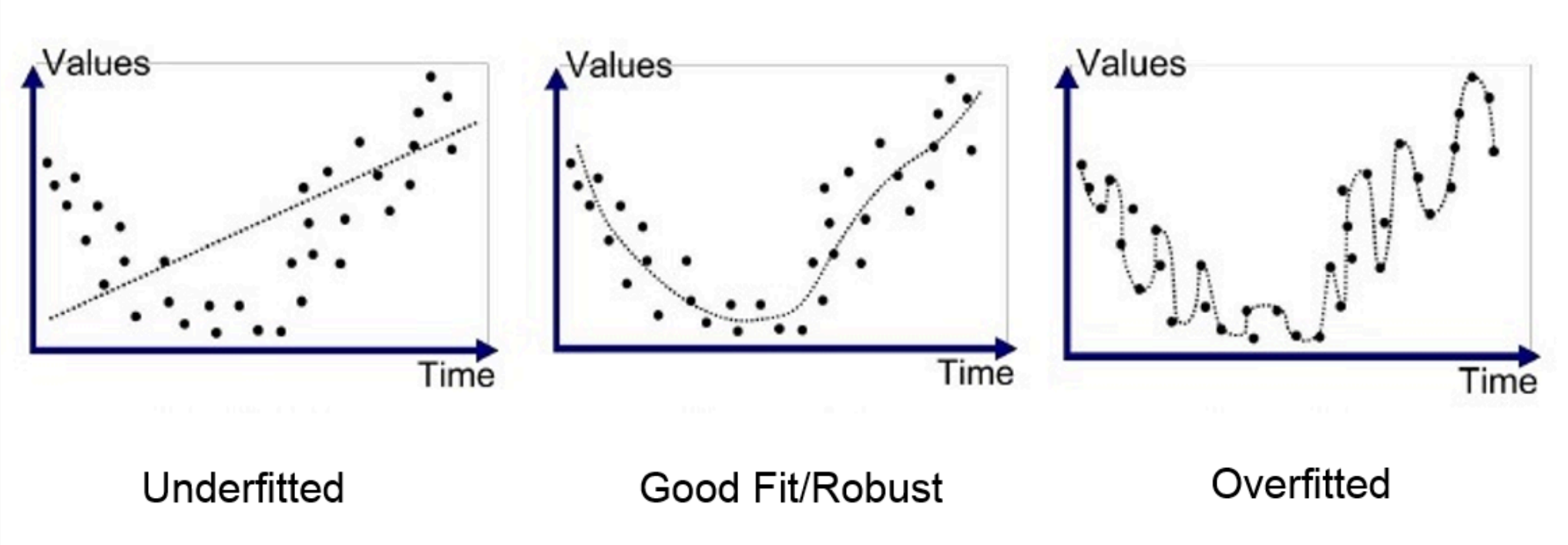
В чём проблема?

Как мы уже ранее убедились, глубокие нейронные сети – это выдающиеся и дорогие модели, которые способны извлекать из данных сложные и нетривиальные паттерны. Однако за это есть своя плата – большой риск переобучения.

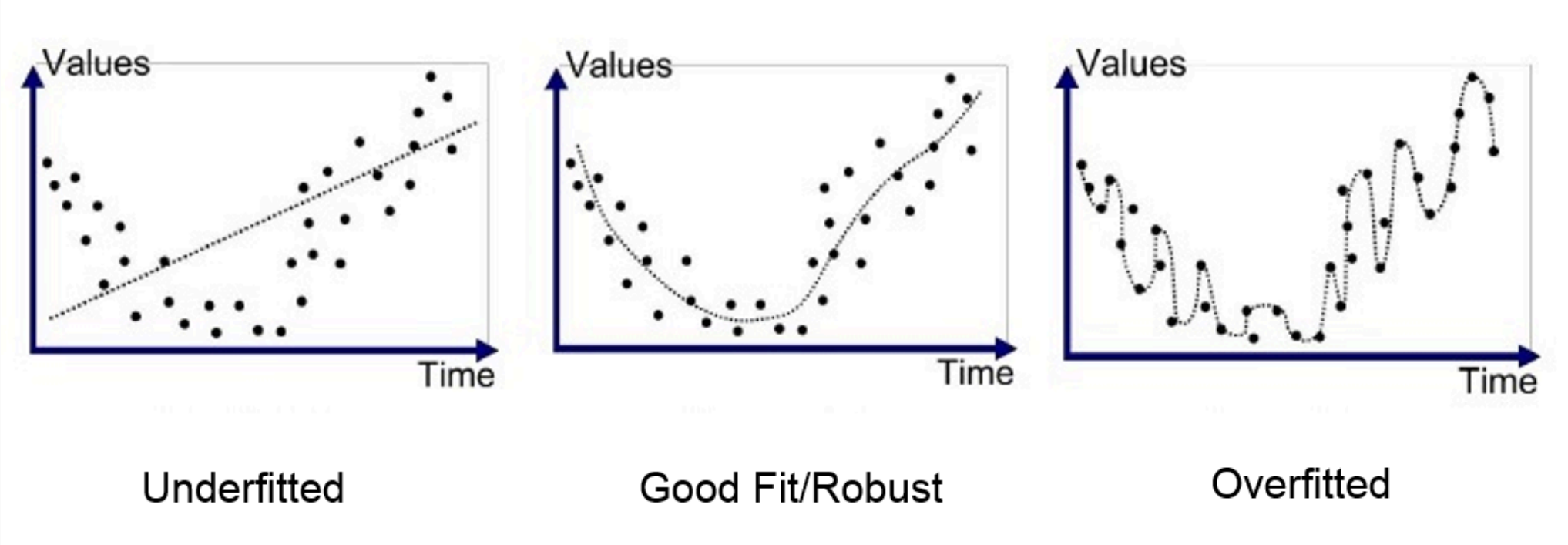
В прошлом семестре мы обсуждали понятие переобучения, когда модель не находила закономерность в данных, а просто старалась их выучить.

В чём проблема?

&&



В чём проблема? &&



Проблема заключается в том, что при добавлении новых данных, левая и правая функции будут не в состоянии *объяснить* эти данные.

Как мы это решали раньше? (Weight decay) &&

Ранее мы изменяли нашу функцию потерь таким образом, чтобы она штрафовала больше (т.е. выдала большее значение лосса) в случае, когда L1 / L2 норма весов слишком большая:

Как мы это решали раньше? (Weight decay) &&

Ранее мы изменяли нашу функцию потерь таким образом, чтобы она штрафовала больше (т.е. выдала большее значение лосса) в случае, когда L1 / L2 норма весов слишком большая:

$$\frac{1}{\ell}\sum_{i=1}^{\ell}(\hat{y}_i - y_i)^2 + \alpha\ \|w\|_2^2, \text{где} \quad \alpha \in [0, 1]$$

Как мы это решали раньше? (Weight decay) &&

Ранее мы изменяли нашу функцию потерь таким образом, чтобы она штрафовала больше (т.е. выдала большее значение лосса) в случае, когда L1 / L2 норма весов слишком большая:

$$\frac{1}{\ell}\sum_{i=1}^{\ell}(\hat{y}_i - y_i)^2 + \alpha\ \|w\|_2^2, \text{где} \quad \alpha \in [0, 1]$$

В случае с нейронными сетями это также уместно, однако не всегда. В некоторых случаях подходят другие методы, о которых мы поговорим далее.

Как бы ещё решить эту проблему?

Как бы ещё решить эту проблему? &&

Как мы знаем, ансамбли решали эту проблему на уровне архитектуры: они обучали много маленьких моделей, которые точно не смогут переобучиться, а выходы всех этих моделей усредняли (в случае регрессии) или “проводили голосование” (в случае классификации).

Как бы ещё решить эту проблему? &&

Как мы знаем, ансамбли решали эту проблему на уровне архитектуры: они обучали много маленьких моделей, которые точно не смогут переобучиться, а выходы всех этих моделей усредняли (в случае регрессии) или “проводили голосование” (в случае классификации).

Однако обучение глубоких сетей – сама по себе дорогостоющая операция, не то что обучать несколько десятков таких моделей…

Метод, решающий обе проблемы

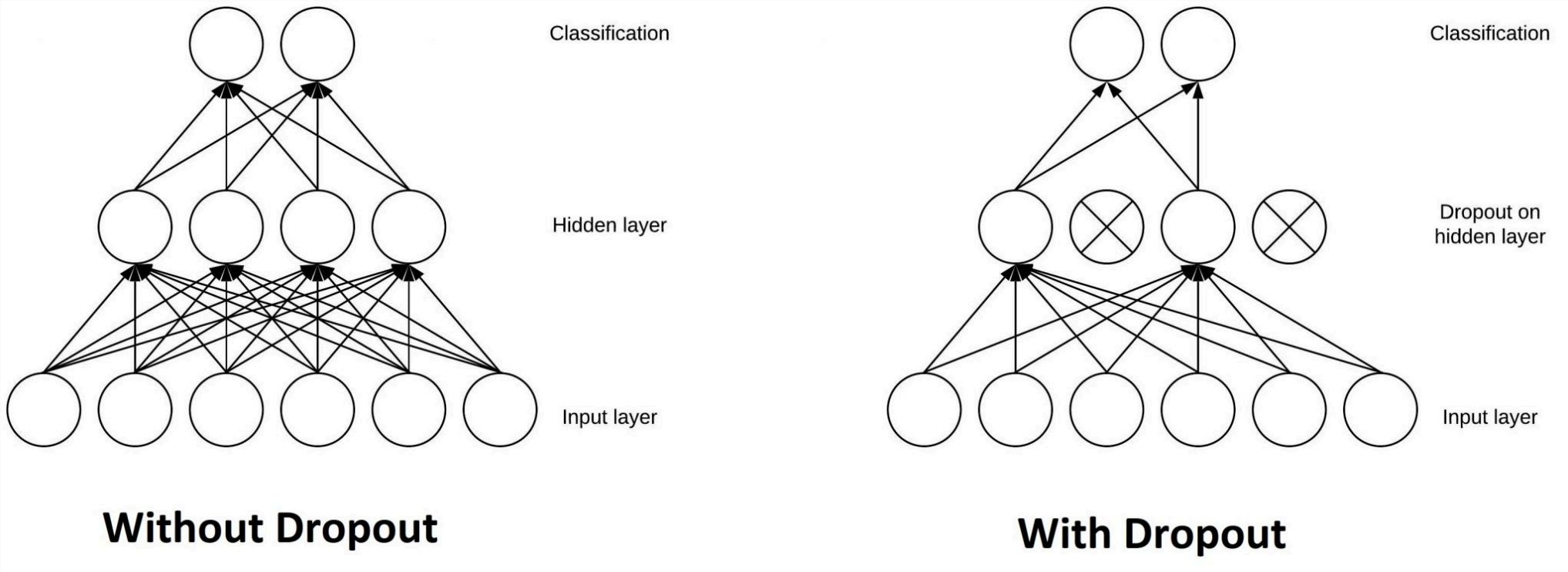
&&

Выходит, что у нас есть проблема с переобучением и со стоимостью обучения. Обе эти проблемы решает техника **Dropout**.

Dropout применяется к каждому слою по отдельности и имеет единственный параметр – вероятность, с которой нейрон “выпадет” из процесса обучения.

Метод, решающий обе проблемы

&&



Метод, решающий обе проблемы

&&

В своей сущности, применение Dropout с $p = 0.5$ в моделе с n нейронами означает, что мы обучаем 2^n моделей, которые делят между собой одинаковые веса (shared weights models).

Метод, решающий обе проблемы

&&

В своей сущности, применение Dropout с $p = 0.5$ в моделе с n нейронами означает, что мы обучаем 2^n моделей, которые делят между собой одинаковые веса (shared weights models).

А учитывая, что некоторые нейроны полностью выпадают из обучения, то и количество весов становится меньше $\Rightarrow$ такие модели становится сложнее переобучить.

Таким образом, Dropout может помочь решить сразу обе возникшие проблемы. Однако взамен такие модели требуют больше итераций для сходимости.

А что делать на тесте?

&&

А что делать на тесте? &&

На самом деле, ничего делать не нужно.

Официальная статья указывает, что отключение Dropout на тестовой выборке означает аппроксимацию усреднения тех самых 2^n моделей. Таким образом, нужно просто выключить Dropout и работать с моделью, которая получилась.

А что делать на тесте? &&

На самом деле, ничего делать не нужно.

Официальная статья указывает, что отключение Dropout на тестовой выборке означает аппроксимацию усреднения тех самых 2^n моделей. Таким образом, нужно просто выключить Dropout и работать с моделью, которая получилась.

Однако возникает другая проблема: все нейроны в совокупности могут выдавать числа, которые следующий слой никак “не ожидает”.

Актуальным решением данной проблемы является **Inverted Dropout**: в обучении мы усиливаем активацию оставшихся нейронов пропорционально количеству неактивных нейронов. Соответственно для вероятности p в текущем слое, формула выхода следующая:

Inverted Dropout

&&

Актуальным решением данной проблемы является **Inverted Dropout**: в обучении мы усиливаем активацию оставшихся нейронов пропорционально количеству неактивных нейронов. Соответственно для вероятности p в текущем слое, формула выхода следующая:

$$y_i = \frac{w^T x}{1 - p}$$

Технически, проблема сложности обучения моделей по прежнему осталась, хоть она перестала быть настолько сильной. Разумеется, что если есть возможность сделать эту проблему менее заметной, то стоит ей воспользоваться...

Пробуем регулировать обучение

Пробуем регулировать обучение

&&

Первая мысль: давайте будем прерывать обучение в момент, когда ошибка перестаёт уменьшаться.

Пробуем регулировать обучение

&&

Первая мысль: давайте будем прерывать обучение в момент, когда ошибка перестаёт уменьшаться.

Проблема: оптимизация может выйти на плато, преодолев которое ошибка упадёт ещё сильнее. Если мы прервёмся в начале плато, то никогда не пройдём его и не сможем уменьшить ошибку.

Пробуем регулировать обучение &&

Вторая мысль: давайте будем прерывать обучение в момент, когда ошибка перестаёт уменьшаться в течение n итераций.

Пробуем регулировать обучение &&

Вторая мысль: давайте будем прерывать обучение в момент, когда ошибка перестаёт уменьшаться в течение n итераций.

Проблема: проверять качество модели на обучающей выборке – идея сомнительная, т.к. из раза в раз именно их она видит. Вот бы иметь похожие данные, которые модель при этом никогда не видела...

Пробуем регулировать обучение &&

Третья мысль: давайте будем прерывать обучение в момент, когда ошибка на отложенной выборке перестаёт уменьшаться в течение n итераций.

Пробуем регулировать обучение

&&

Третья мысль: давайте будем прерывать обучение в момент, когда ошибка на отложенной выборке перестаёт уменьшаться в течение n итераций.

Проблема: если ошибка уменьшится на 0.0000001 – это улучшение, или мы топчемся на месте?

Пробуем регулировать обучение &&

Четвёртая мысль: давайте будем прерывать обучение в момент, когда ошибка на отложенной выборке уменьшается на $\Delta L < t$ в течение n итераций.

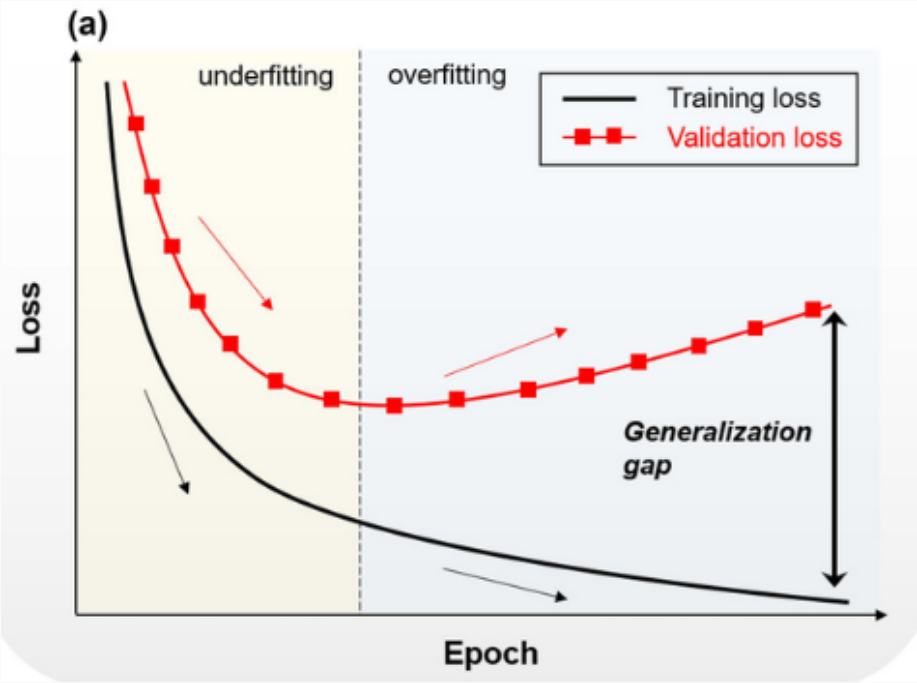
Пробуем регулировать обучение &&

Четвёртая мысль: давайте будем прерывать обучение в момент, когда ошибка на отложенной выборке уменьшается на $\Delta L < t$ в течение n итераций.

Проблем никаких нет, этот метод называется **early stopping** (критерий ранней остановки). Параметр n подбирается извне и называется patience (терпение к неулучшению), t – порог изменения (min delta).

Generalization Gap

&&



Проблема последнего шага

&&

Существует такая довольно явная проблема: если в течение n итераций ошибка не уменьшается (т.е. или не меняется или увеличивается), и мы останавливаем алгоритм, то наилучшая модель с точки зрения функции потерь была n итераций назад. Соответственно нужно брать её.

Существует такая довольно явная проблема: если в течение n итераций ошибка не уменьшается (т.е. или не меняется или увеличивается), и мы останавливаем алгоритм, то наилучшая модель с точки зрения функции потерь была n итераций назад. Соответственно нужно брать её.

Здесь мы познакомились с ещё одной концепцией в обучении моделей – сохранение контрольных точек (checkpointing), когда мы берём не просто последнюю модель, а лучшую согласно значению функции потерь.

Интересное наблюдение

&&

Существует математическое доказательство того, что Early Stopping эквивалентен L2 регуляризации, только в одном случае мы делаем это посредством воздействия на процесс обучения, а в другом – посредством воздействия на функцию потерь.

Нормализация в НС

Ранее мы сталкивались с проблемой, что у нас разные признаки могут находиться в разных масштабах. Для того, чтобы избавиться от этого, из каждого признака вычиталось его среднее и делилось на разброс вокруг среднего:

Нормализация в НС

&&

Ранее мы сталкивались с проблемой, что у нас разные признаки могут находиться в разных масштабах. Для того, чтобы избавиться от этого, из каждого признака вычиталось его среднее и делилось на разброс вокруг среднего:

$$x_{new} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

(ε в знаменателе, чтобы не возникло деления на 0)

В нейронных сетях также без этого никуда: возникают всё те же проблемы со стабильностью обучения.

Представьте себе ситуацию, когда от итерации к итерации входы для второго слоя постоянно изменяют свою природу: в первой итерации среднее этих входов было в районе 0, на второй итерации теперь в районе 10 и т.д.

В нейронных сетях также без этого никуда: возникают всё те же проблемы со стабильностью обучения.

Представьте себе ситуацию, когда от итерации к итерации входы для второго слоя постоянно изменяют свою природу: в первой итерации среднее этих входов было в районе 0, на второй итерации теперь в районе 10 и т.д.

Появляется разумное предположение, что в таких условиях слой просто не поймёт, к каким данным ему нужно быть готовым и как к ним правильно подобрать W.

Это одна из фундаментальных проблем нейронных сетей, и она называется **Internal Covariate Shift** (внутренний сдвиг).

Это одна из фундаментальных проблем нейронных сетей, и она называется **Internal Covariate Shift** (внутренний сдвиг).

Для решения такой проблемы отлично подходит стандартизация, однако в НС применяется более продвинутый вариант – **BatchNorm**

Помимо уже известной нам операции, было бы интересно добавить способность модели самостоятельно выбирать, нужна ли на этом слое стандартизация или нет.

Помимо уже известной нам операции, было бы интересно добавить способность модели самостоятельно выбирать, нужна ли на этом слое стандартизация или нет.

Для этого достаточно составить уравнение с обучаемыми параметрами:

$$y_i = \gamma \hat{x_i} + \beta, \text{где} \quad \hat{x_i} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

, то есть γ и β подбираются моделью.

Помимо уже известной нам операции, было бы интересно добавить способность модели самостоятельно выбирать, нужна ли на этом слое стандартизация или нет.

Для этого достаточно составить уравнение с обучаемыми параметрами:

$$y_i = \gamma \hat{x_i} + \beta, \text{где} \quad \hat{x_i} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

, то есть γ и β подбираются моделью.

В случае, если в ходе обучения модель не посчитает нужным применять какую-то стандартизацию, то она просто установит $\gamma = \sqrt{\sigma^2 + \varepsilon}$ и $\beta = \mu$. Такой случай называется **Identity Transform** (идентичное преобразование), т.к. BatchNorm хоть и применяется, но на деле ничего не изменяет.

Обычно BatchNorm применяется сразу после подсчёта линейной комбинации (т.е. до применения функции активации), однако иногда неплохо работает применение BatchNorm после функции активации, если таковой является ReLU.

Обычно BatchNorm применяется сразу после подсчёта линейной комбинации (т.е. до применения функции активации), однако иногда неплохо работает применение BatchNorm после функции активации, если таковой является ReLU.

Такой подход к решению проблемы сильно зависит от размера батча – если он маленький, то толку от него не будет. Поэтому иногда берут среднее и стандартное отклонение не для признаков, а для каждого объекта.

Обычно BatchNorm применяется сразу после подсчёта линейной комбинации (т.е. до применения функции активации), однако иногда неплохо работает применение BatchNorm после функции активации, если таковой является ReLU.

Такой подход к решению проблемы сильно зависит от размера батча – если он маленький, то толку от него не будет. Поэтому иногда берут среднее и стандартное отклонение не для признаков, а для каждого объекта.

Такой подход называется **LayerNorm** и он хорошо себя показывает даже для батча, в котором находится только 1 объект.

Что происходит на тесте? &&

На тесте BatchNorm и LayerNorm остаются активными и применяются, однако в случае BatchNorm посчитанные ранее среднее и отклонение не пересчитываются: модель “хранит” их значения из процесса обучения. LayerNorm работает точно также, как и при обучени.